

AI Inventory Agent - Viva / Oral Exam Questions & Answers

Section 1: Project Overview and Architecture

Q1: What is the AI Inventory Agent?

A: The AI Inventory Agent is an intelligent inventory management system that combines a traditional CRUD backend with AI-powered analytics. It tracks products, suppliers, inventory movements, sales, and purchases, while providing demand forecasting, reorder suggestions, anomaly detection, and stock health analysis through a Python-based AI microservice.

Q2: What is the overall architecture of the project?

A: The project follows a three-tier architecture with three separate services:

1. **Frontend** (HTML/JavaScript) - Client-side UI served on port 3000
2. **Backend** (Spring Boot) - REST API server on port 8080
3. **AI Service** (Python/Flask) - Prediction microservice on port 5000

The backend communicates with MySQL for persistence and calls the Python AI service for predictions. The frontend makes REST API calls to the Spring Boot backend.

Q3: What technologies are used in this project?

A:

- **Backend:** Java 17, Spring Boot 3.2.5, Spring Data JPA, Spring Security, MySQL 8.0
- **Frontend:** HTML5, CSS3, JavaScript (vanilla), Chart.js for visualizations
- **AI Service:** Python 3.9+, Flask, pandas, scikit-learn, numpy
- **Build Tool:** Maven 3.8+
- **Authentication:** JWT (jjwt library v0.12.3)
- **API Docs:** SpringDoc OpenAPI (Swagger UI)
- **Utilities:** Lombok for boilerplate reduction

Q4: Why did you choose Spring Boot over other frameworks?

A: Spring Boot was chosen because:

- It provides excellent integration with JPA/Hibernate for database operations
- Spring Security offers robust authentication and authorization
- Auto-configuration reduces boilerplate setup
- Built-in support for REST APIs, validation, and exception handling
- Large ecosystem with extensive documentation and community support
- Easy to integrate with external services like the Python AI microservice

Q5: How does the frontend communicate with the backend?

A: The frontend communicates with the backend through RESTful HTTP requests. It sends JSON payloads to the Spring Boot API endpoints and receives JSON responses. Authentication is handled by sending a JWT token in the `Authorization` header (`Bearer <token>`) with each request. CORS is configured to allow cross-origin requests from the frontend.

Section 2: Spring Boot Features

Q6: What Spring Boot starters are used in this project?

A:

- `spring-boot-starter-web` - For building RESTful web services and MVC
- `spring-boot-starter-data-jpa` - For JPA/Hibernate database access
- `spring-boot-starter-security` - For authentication and authorization
- `spring-boot-starter-validation` - For Bean Validation (Jakarta Validation)

Q7: Explain the role of `@RestController` and `@RequestMapping`.

A: `@RestController` is a specialized version of `@Controller` that combines `@Controller` and `@ResponseBody`. It ensures that return values of handler methods are written directly to the HTTP response body (as JSON) rather than being resolved as view names. `@RequestMapping("/api/products")` maps all handler methods in the class to the base path `/api/products`, establishing the URL prefix for all endpoints in that controller.

Q8: What is the purpose of `@RestControllerAdvice`?

A: `@RestControllerAdvice` (used in `GlobalExceptionHandler`) is a combination of `@ControllerAdvice` and `@ResponseBody`. It allows defining `@ExceptionHandler` methods that apply to all controllers globally. When an exception is thrown during request processing, the matching handler method intercepts it and returns a structured JSON error response with appropriate HTTP status codes, instead of exposing stack traces to clients.

Q9: How does the application handle validation?

A: Validation is handled through Jakarta Bean Validation annotations on DTO classes (e.g., `@NotBlank`, `@NotNull`, `@Positive`, `@Size`, `@Email`, `@Min`). When a `@RequestBody` is annotated with `@Valid`, Spring automatically validates the incoming data. If validation fails, a `MethodArgumentNotValidException` is thrown, which is caught by the `GlobalExceptionHandler` and returned as a 400 Bad Request with field-level error messages.

Q10: What is the purpose of Lombok in this project?

A: Lombok is used to reduce boilerplate code through compile-time annotation processing:

- `@Data` generates getters, setters, `toString`, `equals`, and `hashCode`
- `@NoArgsConstructor` and `@AllArgsConstructor` generate constructors
- `@Builder` generates the Builder design pattern for object creation
- This keeps entity and DTO classes concise while maintaining full functionality.

Q11: How are `@PrePersist` and `@PreUpdate` lifecycle callbacks used?

A: These JPA entity lifecycle callbacks automatically manage timestamps:

- `@PrePersist (onCreate())` sets `createdAt` and `updatedAt` to `LocalDateTime.now()` before the entity is first saved to the database.
- `@PreUpdate (onUpdate())` updates `updatedAt` to the current time before any update operation.
- They also handle business logic like auto-calculating `totalPrice = unitPrice * quantity` in the `Sale` entity and `totalCost = unitCost * quantity` in the `Purchase` entity.

Q12: What is the role of `application.properties` in the project?

A: `application.properties` configures the Spring Boot application:

- Server port (8080)
- MySQL datasource URL, credentials, and HikariCP pool settings
- JPA/Hibernate dialect, DDL strategy, and SQL formatting
- JWT secret key and token expiration
- AI service URL for microservice communication
- SpringDoc/Swagger documentation paths

- Logging levels and Actuator endpoints
-

Section 3: JPA/Hibernate and Database Design

Q13: How many tables does the database have and what are they?

A: The database has 6 tables:

1. `users` - System users and authentication
2. `products` - Product catalog
3. `suppliers` - Vendor information
4. `inventory_movements` - Audit log of all stock changes (IN/OUT)
5. `sales` - Customer sales records
6. `purchases` - Supplier purchase orders

Q14: Explain the relationship between products and `inventory_movements`.

A: This is a One-to-Many relationship. One product can have many inventory movement records. The `inventory_movements` table has a `product_id` foreign key referencing `products.id`. Stock levels are not stored as a column but computed dynamically: $\text{Current Stock} = \text{SUM}(\text{IN movements}) - \text{SUM}(\text{OUT movements})$. This approach provides a complete audit trail of all stock changes.

Q15: Why is stock level not stored as a column in the products table?

A: Storing stock as a computed value from movements provides:

- A complete audit trail of every stock change
- Prevention of data inconsistencies between the stock level and movement history
- Ability to trace exactly when and why stock changed
- Thread-safe updates since each movement is an append-only record
- Historical stock analysis at any point in time

Q16: What is the purpose of the `is_active` field in the products table?

A: The `is_active` field implements a **soft delete** pattern. Instead of physically removing a product record (which would break foreign key references in sales, purchases, and inventory_movements), the product is deactivated by setting `is_active = false`. This preserves referential integrity while hiding the product from active listings.

Q17: How does Hibernate's `ddl-auto=update` work?

A: When `spring.jpa.hibernate.ddl-auto=update` is set, Hibernate compares the current entity model with the existing database schema on startup and applies any necessary changes:

- It will **add** new tables, columns, and constraints that are in the entity but missing from the database.
- It will **not drop** tables, remove columns, or modify existing column types.
- This is suitable for development but risky in production. In production, migration tools like Flyway or Liquibase are recommended for controlled, versioned schema changes.

Q18: What is the significance of `FetchType.LAZY` on `@ManyToOne` relationships?

A: `FetchType.LAZY` means the related entity (e.g., `Product`) is not loaded from the database until it is explicitly accessed. This prevents the **N+1 query problem** -- when loading a list of 100 sales, without LAZY loading, Hibernate would execute 101 queries (1 for the sales list + 100 for each product). With LAZY loading, only the initial query runs, and related data is fetched on-demand via a separate query when accessed.

Q19: What is the difference between `findBySku` and `findBySkuContaining`?

A: `findBySku` (used in `ProductRepository`) performs an **exact match** lookup on the SKU column -- it returns a product only if the SKU string matches exactly. `findBySkuContaining` would perform a **partial match** (LIKE '%query%'). The project uses exact matching for SKU lookups because SKUs are unique identifiers, not search terms.

Q20: How are custom queries implemented in Spring Data JPA?

A: Custom queries are implemented using the `@Query` annotation on repository interface methods. For example:

```
@Query("SELECT COALESCE(SUM(im.quantity), 0) FROM InventoryMovement im  
        WHERE im.product.id = :productId AND m.movementType = 'IN'")  
Integer findTotalInByProductId(@Param("productId") Long productId);
```

This uses JPQL (Java Persistence Query Language) which operates on entity objects rather than raw SQL tables. Spring Data JPA automatically implements the method body at runtime.

Section 4: JWT Authentication Flow

Q21: Explain the complete JWT authentication flow.

A:

1. **Registration/Login:** User sends credentials to `/api/auth/register` or `/api/auth/login`.
2. **Token Generation:** The `AuthService` validates credentials, then generates a JWT using HMAC-SHA256 with a secret key and 24-hour expiration.
3. **Token Response:** The JWT is returned to the client in the `AuthResponse` along with user details.
4. **Subsequent Requests:** The client includes the token in the `Authorization: Bearer <token>` header.
5. **Filter Interception:** The `JwtAuthenticationFilter` (a `OncePerRequestFilter`) intercepts every request, extracts the token, validates it, and loads the user from the database.
6. **Security Context:** If valid, a `UsernamePasswordAuthenticationToken` is created and set in the `SecurityContextHolder`, making the user authenticated for the duration of the request.
7. **Protected Endpoints:** Any endpoint not matching `/api/auth/**` or `/api/public/**` requires authentication.

Q22: How is the JWT secret key used?

A: The JWT secret key (`jwt.secret` in `application.properties`) is a Base64-encoded string used to generate an HMAC-SHA256 signing key via `Keys.hmacShaKeyFor(jwtSecret.getBytes())`. This key is used both to sign tokens during generation (`AuthService.generateToken()`) and to verify tokens during authentication (`JwtUtil.validateToken()`). The same key must be used for both operations.

Q23: What happens if a JWT token expires?

A: The JWT contains an `expiration` claim set to 24 hours from issuance. When the `JwtAuthenticationFilter` processes a request with an expired token:

1. `jwtUtil.extractUsername(jwt)` may still return the username
2. `jwtUtil.validateToken(jwt, userDetails)` will return `false` because the token is expired
3. No authentication is set in the `SecurityContextHolder`
4. Spring Security rejects the request with a 401 Unauthorized response

Q24: Why is session management set to STATELESS?

A: `SessionCreationPolicy.STATELESS` tells Spring Security not to create or use HTTP sessions. This is standard for JWT-based authentication because:

- Each request is self-contained with the token in the header
- No server-side session storage is needed, improving scalability
- Multiple server instances can serve requests without session replication
- It's simpler to scale horizontally

Q25: How are passwords stored in the database?

A: Passwords are stored using **BCrypt** hashing via `BCryptPasswordEncoder`. When a user registers, the plain-text password is encoded: `passwordEncoder.encode(request.getPassword())`. During login, the provided password is verified against the stored hash: `passwordEncoder.matches(request.getPassword(), user.getPassword())`. BCrypt is a strong adaptive hashing algorithm that includes salting to prevent rainbow table attacks.

Q26: What does CSRF disabling mean and why is it done?

A: CSRF (Cross-Site Request Forgery) protection is disabled via `csrf(AbstractHttpConfigurer::disable)`. This is standard for stateless REST APIs because:

- CSRF attacks exploit cookie-based session authentication
- JWT tokens are sent in headers, not cookies, so browsers don't automatically include them
- Each request must explicitly include the token, making CSRF attacks ineffective
- Enabling CSRF would require additional token management that's unnecessary for API-only backends

Q27: Which endpoints are publicly accessible without authentication?

A: The following are publicly accessible:

- `POST /api/auth/register` - User registration
- `POST /api/auth/login` - User login
- `GET/POST/PUT/DELETE/OPTIONS /api/public/**` - Any public API path
- `OPTIONS /**` - CORS preflight requests

All other endpoints require a valid JWT token in the `Authorization` header.

Section 5: AI/ML Integration

Q28: How does the AI service communicate with the Spring Boot backend?

A: The Spring Boot backend acts as a proxy between the frontend and the Python AI service. When the frontend calls an AI endpoint (e.g., `GET /api/ai/forecast/1`), the `AIService` in Spring Boot makes an HTTP request to the Python Flask service running at `http://localhost:5000` (configured via `ai.service.url`). The Python service processes the request, runs ML algorithms, and returns JSON which is forwarded to the frontend.

Q29: What AI capabilities does the system provide?

A:

1. **Demand Forecasting** (/ai/forecast/{productId}) - Predicts future demand based on historical sales data using time-series analysis
2. **Reorder Suggestions** (/ai/reorder-suggestions) - Recommends which products to reorder, how much, and how urgently
3. **Anomaly Detection** (/ai/anomalies) - Identifies unusual patterns like sudden demand spikes or stock discrepancies
4. **Stock Health Analysis** (/ai/stock-health) - Provides an overall health score for inventory management
5. **Natural Language Queries** (/ai/query) - Allows users to ask questions about inventory data in plain English

Q30: How does the demand forecasting work?

A: The Python AI service collects historical sales data for a product, applies time-series analysis (likely using statistical methods or machine learning models like ARIMA or Prophet), and generates predictions for:

- Next week, next month, and next quarter demand
- Trend direction (increasing/decreasing/stable)
- Confidence scores for each prediction
- Seasonality patterns and peak periods

Q31: How does the system detect anomalies?

A: The anomaly detection system monitors several metrics:

- **Stock discrepancies** - Compares expected vs. actual stock levels
- **Unusual sale volumes** - Detects when daily sales significantly deviate from rolling averages
- **Demand spikes** - Identifies sudden increases in product demand
- Each anomaly is classified by type and severity (HIGH/MEDIUM/LOW)

Q32: How are reorder suggestions generated?

A: The reorder suggestion algorithm considers:

- Current stock level for each product
- Product's reorder level threshold
- Historical demand patterns and trends
- Lead time estimates from suppliers
- Predicted stockout dates based on current consumption rates
- Each suggestion includes urgency level (HIGH/MEDIUM/LOW), suggested order quantity, and estimated cost.

Q33: What is the purpose of the natural language query endpoint?

A: The `/api/ai/query` endpoint allows users to ask questions about their inventory data in plain English (e.g., "What are the top 5 selling products this month?"). The AI service processes the natural language query, translates it into structured data operations, retrieves the relevant data, and returns both a human-readable answer and the underlying structured data.

Q34: What machine learning libraries does the AI service use?

A: Based on the project requirements:

- **pandas** - Data manipulation and analysis of sales/inventory data
- **scikit-learn** - Machine learning algorithms for forecasting and anomaly detection
- **numpy** - Numerical computations
- **Flask** - Web framework for the REST API serving predictions

Section 6: Frontend-Backend Communication

Q35: How does CORS work in this application?

A: CORS (Cross-Origin Resource Sharing) is configured in `CorsConfig.java` which implements `WebMvcConfigurer`:

```
registry.addMapping("/api/**")
    .allowedOrigins("*")
    .allowedMethods("GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS")
    .allowedHeaders("*");
```

This allows any origin to access any `/api/**` endpoint using standard HTTP methods. This is necessary because the frontend (port 3000) and backend (port 8080) run on different ports, which constitutes a cross-origin request.

Q36: What HTTP methods does the API support?

A: The API supports:

- **GET** - Reading data (products, sales, purchases, dashboard stats, AI predictions)
- **POST** - Creating new resources (products, sales, purchases, inventory movements, user registration)
- **PUT** - Updating existing resources (product updates, purchase status changes)
- **DELETE** - Soft-deleting resources (deactivating products)
- **OPTIONS** - CORS preflight requests (permitted globally)

Q37: How does the frontend handle authentication state?

A: The frontend stores the JWT token (typically in `localStorage` or a cookie) after successful login. For every subsequent API request, the token is included in the `Authorization` header. The JavaScript code in `auth.js` and `common.js` manages login state, token storage, and logout (token removal).

Q38: What format does the API use for request and response bodies?

A: The API uses **JSON** (JavaScript Object Notation) for all request and response bodies. The `Content-Type: application/json` header is used for POST and PUT requests. Date fields use ISO 8601 format (`YYYY-MM-DDTHH:MM:SS`). Decimal values use standard JSON number format with two decimal places.

Section 7: Security Measures

Q39: What security measures are implemented in the application?

A:

1. **JWT Authentication** - Stateless token-based auth with 24-hour expiry
2. **BCrypt Password Hashing** - Passwords are never stored in plain text
3. **Role-Based Access Control** - Users have roles (USER, MANAGER, ADMIN) for authorization
4. **Input Validation** - Jakarta Bean Validation on all DTOs prevents malformed data
5. **SQL Injection Prevention** - JPA/Hibernate parameterized queries prevent SQL injection
6. **CORS Configuration** - Controlled cross-origin access
7. **CSRF Disabled** - Appropriate for stateless REST APIs
8. **Global Exception Handling** - Prevents stack trace leakage to clients
9. **Soft Deletes** - Data is never permanently lost

Q40: How does the application prevent SQL injection?

A: SQL injection is prevented through:

- **JPA/Hibernate ORM** - All database queries are parameterized by default
- **JPQL** - Custom queries use named parameters (`:productId`) which are safely bound
- **Repository Methods** - Spring Data JPA auto-generates parameterized queries from method names
- No raw SQL strings with concatenation are used anywhere in the codebase

Q41: What is the role of `GlobalExceptionHandler`?

A: The `GlobalExceptionHandler` (annotated with `@RestControllerAdvice`) provides centralized exception handling across all controllers:

- `ResourceNotFoundException` -> 404 Not Found with descriptive message
- `MethodArgumentNotValidException` -> 400 Bad Request with field-level validation errors
- `Exception` (catch-all) -> 500 Internal Server Error Each response includes an error type, message, and timestamp, preventing sensitive implementation details from leaking to clients.

Q42: How are API keys and secrets managed?

A: Sensitive configuration values are stored in `application.properties`:

- JWT secret is stored as a Base64-encoded string (`jwt.secret`)
 - Database credentials are stored as plain properties (in production, these should be externalized via environment variables or a vault)
 - The AI service URL is configurable
 - For production deployment, these should be moved to environment variables or a secrets management service.
-

Section 8: Scalability Considerations

Q43: How can this application be scaled?

A:

1. **Horizontal Scaling** - Multiple Spring Boot instances behind a load balancer (stateless JWT auth enables this)
2. **Database Scaling** - Read replicas for MySQL, connection pooling via HikariCP
3. **Microservice Decomposition** - The AI service is already separated; other services can be split
4. **Caching** - Redis or Ehcache for frequently accessed data (product catalog, dashboard stats)
5. **Message Queues** - RabbitMQ/Kafka for async processing of inventory movements and sales
6. **CDN** - For serving static frontend assets

Q44: What is the role of HikariCP in this application?

A: HikariCP is the default JDBC connection pool used by Spring Boot (configured in `application.properties`):

- `maximum-pool-size=20` - Up to 20 concurrent database connections
- `minimum-idle=5` - At least 5 idle connections maintained
- `idle-timeout=300000` - Connections idle for 5 minutes are released
- `max-lifetime=1200000` - Connections are recycled after 20 minutes This ensures efficient database connection management without creating/destroying connections for each request.

Q45: What are the limitations of the current architecture?

A:

1. **Single database instance** - No replication or failover configured
 2. **Synchronous AI calls** - AI service calls block the request thread
 3. **No caching layer** - Repeated queries hit the database every time
 4. **No pagination** - Some list endpoints return all records without pagination
 5. **In-memory stock calculation** - Stock is computed on-the-fly, which could be slow with many movements
 6. **No API rate limiting** - No protection against request flooding
 7. **CORS allows all origins** - Should be restricted in production
-

Section 9: Testing Approach

Q46: What testing frameworks are used?

A:

- **JUnit 5** (via `spring-boot-starter-test`) - Unit and integration testing
- **Spring Boot Test** - Testing Spring context loading and controller behavior
- **Spring Security Test** - Testing security configurations and authentication flows
- These are included as Maven dependencies with `<scope>test</scope>`.

Q47: What types of tests would you write for this application?

A:

1. **Unit Tests** - Test individual service methods in isolation using mocks (Mockito)
2. **Repository Tests** - Test database queries using `@DataJpaTest` with an embedded database
3. **Controller Tests** - Test REST endpoints using `@WebMvcTest` with mocked services
4. **Integration Tests** - Test the full request flow from controller through service to repository
5. **Security Tests** - Verify JWT authentication, role-based access, and endpoint protection
6. **AI Service Tests** - Test the Python ML model accuracy and API response format

Q48: How would you test the JWT authentication?

A: Using Spring Security Test:

1. Mock the `JwtAuthenticationFilter` behavior
2. Test that unauthenticated requests to protected endpoints return 401
3. Test that requests with valid tokens succeed
4. Test that requests with expired or invalid tokens are rejected
5. Test that public endpoints (`/api/auth/**`) work without tokens
6. Use `@WithMockUser` for simulating authenticated users in controller tests

Section 10: Challenges and Solutions

Q49: What was the biggest challenge in building this system?

A: One major challenge was designing the stock level tracking system. Initially, storing stock as a simple column seemed easier, but it couldn't provide an audit trail. The solution was to derive stock from inventory movements (IN/OUT), which provides:

- Complete history of every stock change
- Ability to answer "what was the stock level on date X?"
- Prevention of data inconsistencies
- Clear traceability for discrepancies

Q50: How did you handle the communication between Spring Boot and Python AI service?

A: The Spring Boot backend acts as a proxy, making HTTP calls to the Python Flask service using `RestTemplate` or `WebClient`. Key considerations:

- Error handling: If the AI service is down, the backend returns a graceful error message
- Data transformation: The backend converts database entities into the format expected by the AI service
- Response forwarding: AI predictions are returned to the frontend in a consistent JSON format
- Configuration: The AI service URL is externalized in `application.properties`

Q51: How did you handle the N+1 query problem?

A: The N+1 problem was addressed by using `FetchType.LAZY` on all `@ManyToOne` relationships. When loading lists of sales or purchases, the related product and supplier data is not automatically fetched. It is loaded on-demand when accessed in the controller/service layer. For specific use cases where eager loading is needed, `@Query` with JPQL joins can be used.

Q52: How does the system handle concurrent stock updates?

A: Currently, the system handles concurrency through:

- Database-level atomic operations (INSERT for movements)
- The stock calculation is based on SUM operations which are inherently safe
- Creating a sale checks current stock before proceeding

- For production, optimistic locking (`@Version` annotation) or pessimistic locking should be added to prevent race conditions during high-concurrency scenarios.

Q53: What would you improve if you had more time?

A:

1. Add optimistic locking with `@Version` for concurrent updates
2. Implement API rate limiting and throttling
3. Add Redis caching for frequently accessed data
4. Implement pagination for all list endpoints
5. Add comprehensive unit and integration tests
6. Set up CI/CD pipeline with automated testing
7. Use Flyway for database migration management
8. Add WebSocket support for real-time stock updates
9. Implement proper API versioning (`/api/v1/...`)
10. Externalize all secrets using environment variables or a vault service

Q54: How is the project structured following best practices?

A: The project follows a layered architecture:

```
controller/    -> Handles HTTP requests and responses
service/       -> Business logic and orchestration
repository/    -> Data access (Spring Data JPA interfaces)
entity/        -> JPA entity classes mapped to database tables
dto/           -> Data Transfer Objects for request/response
config/        -> Configuration classes (Security, CORS)
exception/     -> Custom exceptions and global handler
```

This separation ensures single responsibility, testability, and maintainability. Controllers are thin (just HTTP mapping), services contain business rules, and repositories handle data access.

Q55: Explain the complete request lifecycle for creating a sale.

A:

1. Client sends `POST /api/sales` with JWT token and JSON body
2. `JwtAuthenticationFilter` validates the token and sets authentication context
3. `SalesController.createSale()` receives the `SaleRequest` DTO
4. `@Valid` triggers Bean Validation on the request fields
5. `SalesService.createSale()` is called:

- Looks up the product from the database
- Calculates current stock from inventory movements
- Validates sufficient stock is available
- Creates and saves the `Sale` entity
- Creates an `InventoryMovement` with type "OUT" and reference "SALE-{id}"

6. The saved `Sale` is returned as JSON to the client

7. The entire operation runs within a `@Transactional` boundary